[83] Sophie Langer. 2021. Approximating smooth functions by deep neural networks with sigmoid activation function. *Journal of Multivariate Analysis 2021* (2021).

[84] Yoav Shechtman, Amir Beck, and Yonina C Eldar. 2014. GESPAR: Efficient phase retrieval of sparse signals. *IEEE transactions on signal processing* (2014).

[85] Haochuan Li, Jian Qian, Yi Tian, Alexander Rakhlin, and Ali Jadbabaie. 2024. Convex and non-convex optimization under generalized smoothness. *Advances in Neural Information Processing Systems* 36 (2024).

[86] Nikolaos Chatzis, Georgios Smaragdakis, Anja Feldmann, and Walter Willinger. 2013. There is more to IXPs than meets the eye. *SIGCOMM Computer Communication Review* (2013).

[87] Geoff Huston. 2014. How Big is that Network? labs.apnic.net/?p=526

[88] Loqman Salamatian, Calvin Ardi, Vasileios Giotsas, Matt Calder, Ethan Katz-Bassett, and Todd Arnold. [n.d.]. What's in the Dataset? Unboxing the APNIC per AS User Population Dataset. In *IMC 2024*.

[89] Yurii Nesterov et al. 2018. *Lectures on convex optimization*. Vol. 137. Springer.

[90] Gurobi Optimization. 2024. Gurobi Optimizer. https://www.gurobi.com/downloads/

[91] RIPE NCC Staff. 2015. RIPE Atlas: A Global Internet Measurement Network. *Internet Protocol Journal* (2015).

[92] Danilo Giordano, Danilo Cicalese, Alessandro Finamore, Marco Mellia, Maurizio Munafò, Diana Zeaiter Joumblatt, and Dario Rossi. [n.d.]. A First Characterization of Anycast Traffic from Passive Traces. In *TMA 2016*.

[93] Danilo Cicalese, Jordan Augé, Diana Joumblatt, Timur Friedman, and Dario Rossi. [n.d.]. Characterizing IPv4 Anycast Adoption and Deployment. In *CoNEXT 2015*.

[94] Verizon. 2020. Edgecast. https://verizondigitalmedia.com/media-platform/delivery/network/

[95] Matthew Prince. 2013. Load Balancing without Load Balancers. https://blog.cloudflare.com/cloudflares-architecture-eliminating-single-p/

[96] Jing'an Xue, Weizhen Dang, Haibo Wang, Jilong Wang, and Hui Wang. [n.d.]. Evaluating Performance and Inefficient Routing of an Anycast CDN. In *International Symposium on Quality of Service 2019*.

[97] Tobias Flach, Nandita Dukkipati, Andreas Terzis, Barath Raghavan, Neal Cardwell, Yuchung Cheng, Ankur Jain, Shuai Hao, Ethan Katz-Bassett, and Ramesh Govindan. [n.d.]. Reducing web latency: the virtue of gentle aggression. In *SIGCOMM 2013*.

[98] David Wetherall, Abdul Kabbani, Van Jacobson, Jim Winget, Yuchung Cheng, Charles B Morrey III, Uma Moravapalle, Phillipa Gill, Steven Knight, and Amin Vahdat. [n.d.]. Improving Network Availability with Protective ReRoute. In *SIGCOMM 2023*.

[99] Electicity Maps. 2026. Electicity Maps. https://www.electricitymaps.com/data

[100] Sawsan El-Zahr, William Nathan, and Noa Zilberman. [n.d.]. Carbon-Intelligent Content Scheduling in CDNs. In *ANRW 2025*.

[101] Kok-Kiong Yap, Murtaza Motiwala, Jeremy Rahe, Steve Padgett, Matthew Holliman, Gary Baldus, Marcus Hines, Taeeun Kim, Ashok Narayanan, Ankur Jain, et al. [n.d.]. Taking the Edge off with Espresso: Scale, Reliability and Programmability for Global Internet Peering. In *SIGCOMM 2017*.

[102] Raul Landa, Lorenzo Saino, Lennert Buytenhek, and João Taveira Araújo. [n.d.]. Staying Alive: Connection Path Reselection at the Edge. In *NSDI 2021*.

[103] Zheng Zhang, Ming Zhang, Albert G. Greenberg, Y. Charlie Hu, Ratul Mahajan, and Blaine Christian. [n.d.]. Optimizing Cost and Performance in Online Service Provider Networks. In *NSDI 2010*.

[104] Hongqiang Harry Liu, Raajay Viswanathan, Matt Calder, Aditya Akella, Ratul Mahajan, Jitendra Padhye, and Ming Zhang. [n.d.]. Efficiently Delivering Online Services over Integrated Infrastructure. In *NSDI 2016*.

[105] Vytautas Valancius, Bharath Ravi, Nick Feamster, and Alex C Snoeren. [n.d.]. Quantifying the Benefits of Joint Content and Network Routing. In *SIGMETRICS 2013*.

[106] Henry Birge-Lee, Sophia Yoo, Benjamin Herber, Jennifer Rexford, and Maria Apostolaki. [n.d.]. {TANGO}: Secure Collaborative Route Control across the Public Internet. In *NSDI 2024*.

[107] Jiangchen Zhu, Tom Koch, Ilgar Mammadov, Shuyue Yu, Kevin Vermeulen, Matt Calder, Italo Cunha, and Ethan Katz-Bassett. [n.d.]. The New (Pareto) Frontier of Cloud Routing: High Availability, Precise Control, or Configuration Stability-Choose Two. *CoNEXT 2025*.

[108] Minyuan Zhou, Jiaqi Zheng, Congying Wang, Guihai Chen, and Wanchun Dou. [n.d.]. AnyTuner: Optimizing IP Anycast Performance via Strategic BGP Routing Policy. In *ICNP 2025*.

[109] ASM Rizvi, Leandro Bertholdo, João Ceron, and John Heidemann. [n.d.]. Anycast Agility: Network Playbooks to Fight DDoS. In *USENIX Security Symposium 2022*.

[110] Subspace. 2022. Optimize Your Network on Subspace. https://subspace.com/solutions/reduce-internet-latency

[111] INAP. 2022. INAP Network Connectivity. https://inap.com/network/

[112] Akamai. 2022. Akamai Secure Access Service Edge. https://akamai.com/resources/akamai-secure-access-service-edge-sase

[113] Cloudflare. 2022. Argo Smart Routing. https://cloudflare.com/products/argo-smart-routing/

[114] John P Eason, Xueqi He, Richard Cziva, Max Noormohammadpour, Srivatsan Balasubramanian, Satyajeet Singh Ahuja, and Biao Lu. [n.d.]. Hose-based cross-layer backbone network design with Benders decomposition. In *SIGCOMM 2023*.

[115] Yiyang Chang, Chuan Jiang, Ashish Chandra, Sanjay Rao, and Mohit Tawarmalani. 2019. Lancet: Better network resilience by designing for pruned failure sets. (2019).

[116] Chuan Jiang, Sanjay Rao, and Mohit Tawarmalani. [n.d.]. PCF: provably resilient flexible routing. In *SIGCOMM 2020*.

[117] IETF. 2005. Fast Reroute Extensions to RSVP-TE for LSP Tunnels. https://datatracker.ietf.org/doc/html/rfc4090

[118] IETF. 2006. BGP/MPLS IP Virtual Private Networks (VPNs). https://datatracker.ietf.org/doc/html/rfc4364

[119] IETF. 2000. A Framework for Integrated Services Operation over Diffserv Networks. https://datatracker.ietf.org/doc/html/rfc2998

[120] IETF. 2023. Low Latency, Low Loss, and Scalable Throughput (L4S) Internet Service: Architecture. https://datatracker.ietf.org/doc/rfc9330/

[121] Wen Xu and Jennifer Rexford. [n.d.]. MIRO: Multi-Path Interdomain Routing. In *SIGCOMM 2006*.

[122] Sharad Agarwal, Chen-Nee Chuah, Supratik Bhattacharyya, and Christophe Diot. [n.d.]. Impact of BGP Dynamics on Router CPU Utilization. In *PAM 2004*.

A DEFINITION OF TERMS

Table 1 includes a table of common terms and expressions used in Section 3, for ease of reference.

B CONSIDERATIONS FOR REAL-WORLD DEPLOYMENTS

In Section 4 we discuss how we prototype SCULPTOR. Here we give a high-level overview of how a real cloud can practically, safely implement SCULPTOR. The measurement, advertisement, and learning process described in Section 3 is meant to be carried out before any production traffic is shifted to the prefixes SCULPTOR is configuring, as in prior work [1, 16, 47].

SCULPTOR requires latency measurements to each ingress. To obtain these, the Service Provider should direct clients to measure latency towards a test prefix using a measurement system similar to those found in the literature [53, 69–71]. Note that this process *does not involve production traffic*. The Service Provider should then advertise the test prefix to one ingress at a time and record client latencies to the ingress.

With these initial measurements, Service Providers can initialize the learning process described in Section 3.3.3 using test prefixes. Prior work also shows that BGP routing changes minimally disrupt routers [122], so SCULPTOR's learning process using test prefixes does not affect production traffic.

After convergence, SCULPTOR can slowly shift production traffic over to the optimal learned configuration. Note that, since most traffic is already routed along a good path [1, 16, 47, 52], SCULPTOR may only need to shift a small fraction of traffic or do so infrequently [6], mitigating the blast radius of configuration changes. The learning process is very infrequent, as prior work found that advertisements do not need to be updated even after a month [1].

C HEURISTICS FOR FAST COMPUTATION

With sufficient resources, solving Equation (1) using the two pronged approach outlined in Section 3.4 is feasible. Given our advertisement rate limitations (§3.3.1) we wish to compute gradients in tens of minutes. With 100 Monte Carlo samples per gradient and tens of thousands of gradients to compute, we thus need to solve millions of linear programs at each gradient step. Meta recently demonstrated that they could solve millions of linear programs in minutes using their infrastructure [114].

As we do not have these resources, we instead use efficient heuristics to approximately solve each linear program. Our heuristics depend on the structure of our objective function, and so are specific to our choice of average latency (Eq. (2)). During optimization we observed that these heuristics tended to yield accurate estimates for G despite inaccuracies in predicting any individual UG's latency or link's utilization.

C.1 Capacity-Free UG Latency Calculation

Minimizers of Equation (2) balance traffic across links to satisfy capacity constraints. We instead temporarily ignore capacity constraints and assign each UG to their lowest-latency path. This approximation solves two problems at once. First, we do not need to solve a linear program to compute traffic assignments. Second, we can now analytically compute the distribution of our objective function, G, over all possible realizations of $R(A)$. Analytically computing this distribution is useful, since it lets us compute entropy (§3.3.3) and expected value (§3.3.2) without Monte Carlo methods.

To see why we can compute the distribution exactly, notice that the objective function is a composition of functions of the form $min(X, Y)$ and $X + Y$ since we choose the minimum latency path across prefixes for each UG and average these minimums. We compute UG latency distributions for each prefix exactly using our routing preference model, and use analytical methods to efficiently compute the distributions of the corresponding minimums and averages.

C.2 Imposing Capacity Violation Penalties

It could be that such minimum latency assignments lead to congested links. We would like to penalize such advertisement scenarios, and favor those that distribute load more evenly without solving linear programs. Hence, before computing the expected latency, we first compute the probability that links are congested by computing the distribution of link utilizations from the distribution of UG assignments to paths. We then inflate latency for UGs on likely overutilized links.

That is, for each possible outcome of UG assignments to links, we compute link utilizations and note the probability those UGs reach each link. We then accumulate the probability a link is congested as the total probability over all possible scenarios that lead to overutilization. We discourage UGs from choosing paths that are likely congested using a heuristic — we emulate the impact of overloading by inflating latency in this calculation for UGs on those paths proportional to the overutilization factor. After emulating the effect of this overloading, we recompute the expected average UG latency *without changing* UG *decisions*. We do not change UG decisions as doing so could induce an infinite calculation loop if new decisions also lead to overloading. This heuristic penalizes advertisement strategies that would lead to many overutilized links if every user were to choose their lowest latency option. We show an example in Figure 10.

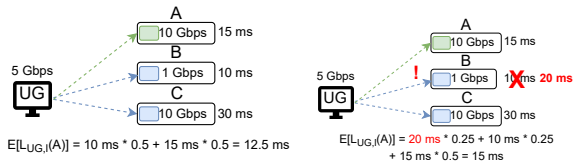
D FURTHER RESULTS

D.1 Steady-State Latency

In Section 5.2.1 we show that SCULPTOR achieves better average latency than other approaches. In addition to quantifying average latency during steady-state, we also compute the fraction of traffic within 10 ms, 50 ms, and 100

Variable	
A	Advertisement configuration over peers and prefixes
w	Traffic allocation to routes
R	Routing function, which maps advertisements to routes. Unknown at first but updated according to measurements
p	Prefix, one of many in a prefix budget (limited due to expenses)
l	Ingress link, Service Providers often have thousands
$c(l)$	Link l capacity
UG	User group, users that route similarly. *e.g.*,/24's or metros
$w(p, \text{UG})$	Amount of traffic from users in UG we direct towards prefix p
G	Objective function, *e.g.*, average latency
$v(\text{UG})$	Traffic volume for users UG
F_l	Failure on link l
β	Tradeoff parameter between minimizing latency and maximum link utilization
M	Maximum link utilization
α_l	Importance parameter for failure on link l, *e.g.*, if one failure is more likely than the other
Expression	
$l = R(A)(p, \text{UG})$	Route (ingress link l) resulting from A, for UG towards p
$\mathcal{L}(\text{UG}, l)$	Latency for UG across l
$G(R(A), w)$	Objective function value under A and w

Table 1: Table of common terms and expressions.



(a) (b)

Figure 10: A UG has a path to two prefixes, green and blue. The green prefix is only advertised via ingress A and the blue prefix is advertised via two ingresses B and C, each of which are equally likely for this UG. If the UG prefers B over C, we assign the client to the blue prefix (10 ms) while if the UG prefers C over B we will assign the client to the green prefix (15 ms). Hence the initial expected latency is the average of 10 ms and 15 ms corresponding to whether this UG prefers ingress B or C (10a). However, ingress B does not have sufficient capacity to handle this UG's traffic and so is congested with 50% probability corresponding to the 50% probability that this user prefers ingress B. We artificially inflate this UG's expected latency (and all other UG's using ingress B) to reflect this possible overutilization (10b).

ms of the One-per-Peering solution during steady-state. Figure 11 shows how SCULPTOR compares more favorably to the One-per-Peering solution than other advertisement strategies.

D.2 Latency During Link/Site Failure

In Section 5.2.1 we show that SCULPTOR offers low latency during link failures. Here we conduct additional evaluations of SCULPTOR during site failures and present further details on link failures. We show that SCULPTOR provides good resilience to site failures on the Internet, and Figure 12 shows similar results for link and site failures in our emulations.

D.2.1 Internet Deployment

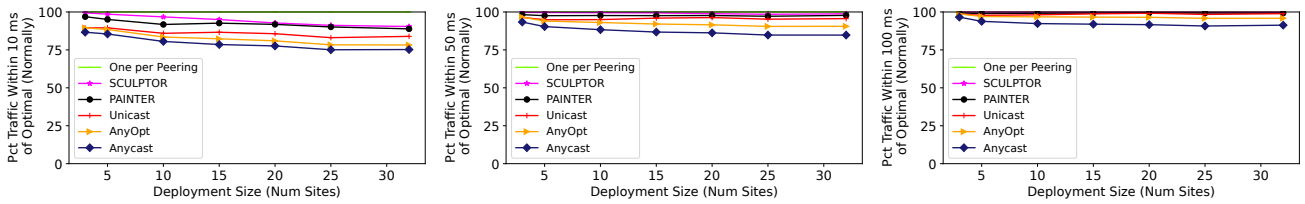
SCULPTOR provides better resilience than other solutions to site failures on the Internet, yielding no congested traffic and on average 18.1 ms worse than One-per-Peering. The next-best solution, unicast, yields 16.1% overloading and, for uncongested traffic, 28.1 ms worse latency than One-per-Peering on average.

D.2.2 Emulations

SCULPTOR provides good performance in emulation.

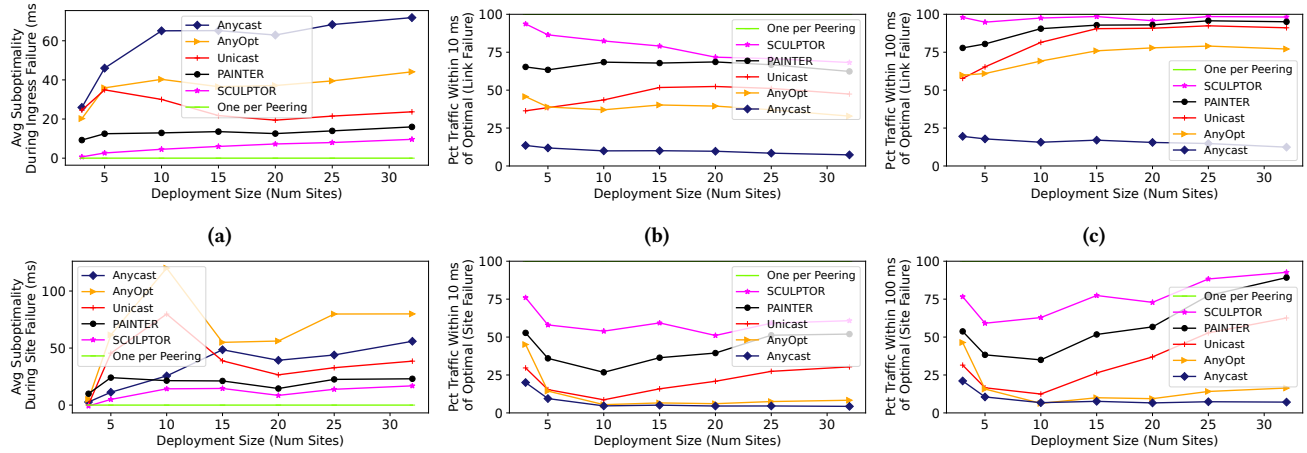
D.3 Infrastructure Utilization Further Results

In Figure 13, we show the shape of our diurnal curve taken from observed diurnal traffic patterns at a Service Provider [21]. We use this diurnal traffic pattern to evaluate how well balances traffic across sites (§5.2.1). Specifically, we linearly interpolate 6 points on the purple curve (Edge Node 1) in Figure 1 of that paper, capturing the essential peaks and troughs.



(a) (b) (c)

Figure 11: SCULPTOR lowers average latency and increases the fraction of traffic within critical thresholds of their optimal latency during normal operation.



(a) (b) (c)

(d) (e) (f)

Figure 12: SCULPTOR lowers average latency and increases the fraction of traffic within critical thresholds of One-per-Peering latency during both link and site failures over many emulated deployments.

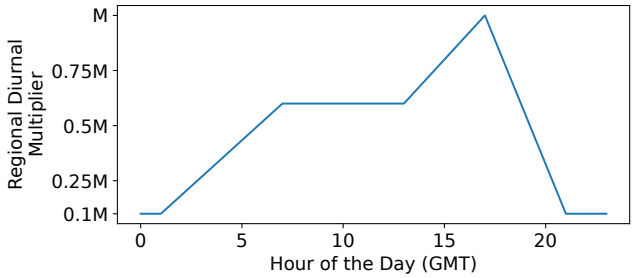


Figure 13: The diurnal traffic shape we apply in Section 5.2.1, taken from prior work [21].